

Rapport code AES simplifié

Projet personnel et éducatif de cryptographie

Maxime Briere
26/12/2025

Table des matières

1. Contexte général
2. Paramètres globaux et entrée/sortie
 - 2.1 Paramètres de chiffrement
 - 2.2 Fichiers d'entrée/sortie
3. Fonctions utilitaires (conversions et padding)
 - 3.1 Lecture de fichier texte
 - 3.2 Texte $\leftrightarrow$ bits (approche pédagogique)
 - 3.2.1 text_to_bits
 - 3.2.2 bits_to_text
 - 3.3 Découpage en blocs
 - 3.4 Padding PKCS#7-like
 - 3.5 Bits $\leftrightarrow$ hex / bytes
 - 3.5.1 bits_to_hex
 - 3.5.2 bytes_to_bits
 - 3.5.3 bits_to_bytes
 - 3.5.4 hex_to_bits
4. Key Schedule : génération des sous-clés
 - 4.1 Rotation gauche
 - 4.2 Génération des sous-clés de round
5. AddRoundKey
6. SubBytes : S-box sur nibbles
 - 6.1 Définition de la S-box
 - 6.2 Fonction sub_bytes
7. ShiftRows : diffusion par réorganisation
8. MixColumns : diffusion inter-nibbles
9. Un round complet de chiffrement
10. Chiffrement complet
11. Déchiffrement bloc par bloc et gestion des bytes
 - 11.1 Fonction decrypt_bits_to_bytes
12. Fonctions de chiffrement et déchiffrement de fichiers
 - 12.1 encrypt_file
 - 12.2 decrypt_file

- 13. Fonctions inverses
 - 13.1 Inverse SubBytes
 - 13.2 Inverse ShiftRows
 - 13.3 Inverse MixColumns
- 14. Déchiffrement texte (wrapper haut niveau)
- 15. Interface interactive (main)
- 16. Vue d'ensemble : pipeline complet

1. Contexte général

Ce programme implémente un **chiffrement par blocs simplifié**, inspiré d'AES, mais sur des blocs et clés de **16 bits** au lieu de 128.

Il applique, comme AES :

- Un **key schedule** (génération de sous-clés de round),
- Des opérations de **confusion** (S-box / SubBytes),
- Des opérations de **diffusion** (ShiftRows, MixColumns),
- Un **AddRoundKey** (XOR avec la sous-clé).

Il travaille sur des **fichiers**, en lisant des bytes, en les convertissant en bits, en chiffrant bloc par bloc, puis en écrivant le résultat en binaire ou en hex.

2. Paramètres globaux et entrée/sortie

```
BLOCK_SIZE = 16
KEY_SIZE = 16
ROUNDS = 4
MASTER_KEY = 0b1010110011001010
INPUT_FILE_TEXT = "./inputs/inputs_AES/input_text_AES.txt"
INPUT_FILE_CRYPT = "./inputs/inputs_AES/input_crypt_AES.txt"
OUTPUT_FILE_CRYPT = "./outputs/AES/output_crypt_AES.txt"
OUTPUT_FILE_DECRYPT = "./outputs/AES/output_decrypt_AES.txt"
```

2.1 Paramètres de chiffrement

Variable	Rôle
BLOCK_SIZE	Taille d'un bloc en bits (16 bits)

KEY_SIZE	Taille de la clé maître (16 bits)
ROUNDS	Nombre de rounds de chiffrement (4)
MASTER_KEY	Clé maître binaire (16 bits)

Le schéma général est donc :

- Chaque **bloc** = 16 bits
- La **clé** = 16 bits
- On applique **4 rounds** successifs pour chiffrer chaque bloc.

2.2 Fichiers d'entrée/sortie

Variable	Rôle
INPUT_FILE_TEXT	Fichier texte clair à chiffrer
INPUT_FILE_CRYPT	Fichier chiffré d'entrée (optionnel)
OUTPUT_FILE_CRYPT	Fichier de sortie chiffré
OUTPUT_FILE_DECRYPT	Fichier de sortie déchiffré

Le main() choisit ces chemins automatiquement, ce qui permet de lancer le programme sans fournir d'arguments.

3. Fonctions utilitaires (conversions & padding)

Ces fonctions forment la “plomberie” I/O pour passer du monde des **fichiers** au monde des **bits**.

3.1 Lecture de fichier texte

```
def read_file(file_path):
    """Lit le contenu d'un fichier et le retourne en tant que chaîne de caractères."""
    with open(file_path, 'r', encoding='utf-8') as f:
        content = f.read()
    return content
```

- **But** : lire un fichier texte en UTF-8 et renvoyer son contenu sous forme de str.
- Cette fonction n'est pas au cœur du chiffrement (les chemins de encrypt_file lisent en binaire), mais elle est cohérente avec l'idée de traiter du texte.

3.2 Texte ↔ bits (approche pédagogique)

Les fonctions `text_to_bits` et `bits_to_text` sont présentes à des fins pédagogiques. Dans le chiffrement réel des fichiers, le programme travaille exclusivement sur des bytes binaires afin d'éviter les problèmes liés à l'encodage UTF-8.

3.2.1 text_to_bits

```
def text_to_bits(text):  
    # Convertit texte → liste de bits  
    bits = []  
    for char in text:  
        bin_char = format(ord(char), '08b')  
        bits.extend(int(b) for b in bin_char)  
    return bits
```

- **Entrée** : text (chaîne Python).
- **Étapes** :
 1. Pour chaque caractère, `ord(char)` donne son code Unicode.
 2. `format(..., '08b')` produit un binaire sur 8 bits (octet).
 3. On décompose cette chaîne '01010100' en une liste de int ([0,1,0,1,0,1,0,0]).
- **Sortie** : une liste de bits (0 ou 1).

Schéma :

```
"AB"  
  
→ 'A' = 65 → "01000001"  
→ 'B' = 66 → "01000010"  
→ [0,1,0,0,0,0,0,1, 0,1,0,0,0,0,1,0]
```

3.2.2 bits_to_text

```
def bits_to_text(bits):  
    # Convertit liste de bits → texte  
    chars = []  
    for i in range(0, len(bits), 8):  
        byte = bits[i:i+8]
```

```

char = chr(int(''.join(str(b) for b in byte), 2))
chars.append(char)
return ''.join(chars)

```

- Recombine les bits par tranches de 8.
- Convertit chaque tranche en entier (int(..., 2)).
- Puis en caractère (chr(...)).
- Concatène les caractères pour former la chaîne finale.

3.3 Découpage en blocs

```

def split_blocks(bits, block_size=BLOCK_SIZE):
    # Découpe bits en blocs de block_size, avec padding si nécessaire
    blocks = []
    for i in range(0, len(bits), block_size):
        block = bits[i:i+block_size]
        if len(block) < block_size:
            block += [0] * (block_size - len(block)) # Padding avec des zéros
        blocks.append(block)
    return blocks

```

- On parcourt les bits par tranches de block_size (= 16).
- Si la dernière tranche est plus petite, on la complète avec des 0.
- Ce padding **n'est pas PKCS#7** : c'est un padding "zéro" pour garantir que chaque bloc a 16 bits. Dans la pratique, le padding PKCS#7-like est utilisé pour le chiffrement. Le padding par zéros dans split_blocks sert uniquement de sécurité structurelle pour garantir des blocs complets.

Schéma :

```

bits : [1,0,1,1,0,0,1,0, 1,1,0,1]    (12 bits)
→ block1 : [1,0,1,1,0,0,1,0,1,1,0,1]  (12 bits)
→ padding : 4 zéros
→ bloc final : [1,0,1,1,0,0,1,0,1,1,0,1,0,0,0,0]

```

3.4 Padding PKCS#7-like

```
def pad_bits_pkcs7(bits, block_size=BLOCK_SIZE):
    # PKCS#7-like padding at byte granularity for block_size (bits)
    if len(bits) % 8 != 0:
        raise ValueError('pad_bits_pkcs7 expects bit length multiple of 8')
    bytes_per_block = block_size // 8
    total_bytes = len(bits) // 8
    rem = total_bytes % bytes_per_block
    if rem == 0:
        pad_bytes = bytes_per_block
    else:
        pad_bytes = bytes_per_block - rem
    pad_val = pad_bytes & 0xFF
    pad_bits = []
    for _ in range(pad_bytes):
        pad_bits.extend([int(b) for b in f"{pad_val:08b}"])
    return bits + pad_bits
```

Idée : on applique un padding de type PKCS#7 mais en travaillant au niveau bits.

- `bytes_per_block = block_size // 8`
Avec `block_size = 16` → `bytes_per_block = 2`. Chaque bloc logique fait **2 octets**.
- On calcule combien de bytes on a :
 - `total_bytes = len(bits) // 8`
 - `rem = total_bytes % bytes_per_block`
- Règle PKCS#7 :
 - Si déjà multiple de la taille de bloc, on ajoute un bloc complet de padding (ici 2 bytes).
 - Sinon, on ajoute juste les bytes nécessaires.

Chaque **byte de padding** vaut la valeur du nombre de bytes ajoutés (`pad_val`).

Exemple :

Si bloc = 4 bytes, il reste 2 bytes → on ajoute 2 bytes de valeur 0x02 0x02.

Ici, taille de bloc = 2 bytes.

Si le message fait 3 bytes → on ajoute 1 byte de valeur 0x01.

Si le message fait 4 bytes → on ajoute 2 bytes de valeur 0x02 0x02.

3.5 Bits ↔ hex / bytes

3.5.1 bits_to_hex

```
def bits_to_hex(bits):  
    # Convertit une liste de bits en chaîne hexadécimale  
  
    if not bits:  
        return ''  
  
    # Pad to full bytes  
    if len(bits) % 8 != 0:  
        bits = bits + [0] * (8 - (len(bits) % 8))  
  
    hex_bytes = []  
    for i in range(0, len(bits), 8):  
        byte = bits[i:i+8]  
        val = int(''.join(str(b) for b in byte), 2)  
        hex_bytes.append(f"{val:02x}")  
  
    return ''.join(hex_bytes)
```

- Regroupe les bits en octets.
- Convertit chaque octet en entier, puis en hex (02x → 2 chiffres hex, zéro-padding).
- Concatène tous les octets hex → chaîne hexadécimale.

3.5.2 bytes_to_bits

```
def bytes_to_bits(b: bytes) -> list:  
    bits = []  
  
    for byte in b:  
        bits.extend([(byte >> i) & 1 for i in range(7, -1, -1)])  
  
    return bits
```

- Pour chaque octet, on extrait les bits du plus significatif (7) au moins significatif (0).
- [(byte >> i) & 1 for i in range(7, -1, -1)] produit 8 bits.

3.5.3 bits_to_bytes

```
def bits_to_bytes(bits: list) -> bytes:  
  
    if len(bits) % 8 != 0:  
        bits = bits + [0] * (8 - (len(bits) % 8))
```



```

ba = bytearray()
for i in range(0, len(bits), 8):
    byte = bits[i:i+8]
    val = int(''.join(str(b) for b in byte), 2)
    ba.append(val)
return bytes(ba)

```

- Complète à un multiple de 8 bits si nécessaire.
- Regroupe les bits en octets, convertit en entiers, puis en bytes.

3.5.4 hex_to_bits

```

def hex_to_bits(hexstr: str) -> list:
    return bytes_to_bits(bytes.fromhex(hexstr))

```

- Prend une chaîne hex, la convertit en bytes, puis en bits.

4. Key Schedule : génération des sous-clés

4.1 Rotation gauche

```

def rotl(x: int, n: int, bits: int = 16) -> int:
    """
    Rotation gauche de n bits sur une taille donnée.
    """
    mask = (1 << bits) - 1
    return ((x << n) & mask) | (x >> (bits - n))

```

Explication :

- $\text{mask} = (1 \ll \text{bits}) - 1$
Pour $\text{bits} = 16 \rightarrow \text{mask} = 0xFFFF$ (16 bits à 1).
Cela permet de tronquer à bits.
- $x \ll n$ décale à gauche.
- $\& \text{mask}$ garde seulement les bits de poids faible.
- $x \gg (\text{bits} - n)$ récupère les bits qui “dépassent” à gauche et qu’on ramène à droite.
- Le $|$ recolle les deux parties.

Schéma (16 bits, rotation de 3) :

$x = \text{abcdefghijklmnop}$ (16 bits)

```
x << 3 = defghijklmnop000
```

```
^^^^^^^^^^^^^^ (on garde 16 bits)
```

```
x >> 13 = abc
```

```
rotation = defghijklmnop000 (tronqué) OR abc → defghijklmnopabc
```

4.2 Génération des sous-clés

```
def generate_round_keys(master_key: int, rounds: int, key_size: int = 16) -> list:
    """
    Génère les sous-clés à partir d'une clé maître.
    ...
    """
    round_keys = []
    motif = 0b1010101010101010 # motif fixe

    current_key = master_key
    for r in range(rounds):
        # rotation gauche de 3 bits
        current_key = rotl(current_key, 3, key_size)

        # XOR avec motif
        current_key ^= motif

        # conversion en liste de bits
        bits_list = [(current_key >> i) & 1 for i in reversed(range(key_size))]
        round_keys.append(bits_list)

    return round_keys
```

Étapes par round :

1. Rotation gauche de 3 bits :

```
current_key = rotl(current_key, 3, key_size)
```

2. XOR avec motif fixe :

`current_key ^= motif`

Le motif 1010... alterne les bits et assure une variation régulière.

3. Conversion en bits :

`[(current_key >> i) & 1 for i in reversed(range(key_size))]`

On récupère les bits de poids fort vers poids faible.

Le résultat est une **liste de rounds sous-clés**, chacune étant une liste de 16 bits.

Tableau conceptuel :

Round	Clé integer après rotl/XOR	Clé en bits (bits_list)
-------	----------------------------	-------------------------

1	<code>current_key_1</code>	<code>[k0, k1, ..., k15]</code>
---	----------------------------	---------------------------------

2	<code>current_key_2</code>	<code>[...]</code>
---	----------------------------	--------------------

...	...	...
-----	-----	-----

5. AddRoundKey

```
def add_round_key(block, round_key):  
    """  
    XOR bit à bit du bloc avec la sous-clé.  
    ...  
    """  
    if len(block) != len(round_key):  
        raise ValueError("Le bloc et la sous-clé doivent avoir la même taille.")  
  
    return [b ^ k for b, k in zip(block, round_key)]
```

- Vérifie que le bloc et la clé ont la même longueur (16 bits).
- `zip(block, round_key)` produit des couples (b, k) pour chaque position.
- `b ^ k` applique le XOR bit à bit.
- La list compréhension reconstruit une nouvelle liste de bits.

Table de vérité du XOR :

b	k	b ^ k
----------	----------	--------------

0	0	0
0	1	1
1	0	1
1	1	0

C'est la même opération qu'AES : une addition de round key mod 2, bit à bit.

6. SubBytes : S-box sur nibbles

6.1 S-box

```
S_BOX = {
    0: 14, 1: 4, 2: 13, 3: 1,
    4: 2, 5: 15, 6: 11, 7: 8,
    8: 3, 9: 10, 10: 6, 11: 12,
    12: 5, 13: 9, 14: 0, 15: 7
}
```

- Entrées : 0 à 15 (4 bits).
- Sorties : permutation de 0 à 15.
- C'est une fonction non linéaire, analogie avec la S-box d'AES (mais beaucoup plus simple).

6.2 Fonction sub_bytes

```
def sub_bytes(block):
    # Substitution nibble par nibble
    new_block = []
    for i in range(0, len(block), 4):
        nibble_bits = block[i:i+4]
        nibble_val = int("".join(map(str, nibble_bits)), 2)
        sub_val = S_BOX[nibble_val]
        sub_bits = [int(b) for b in f"{sub_val:04b}"]
        new_block.extend(sub_bits)

    return new_block
```

Étapes :

1. On parcourt le bloc par **nibbles** (4 bits).
2. On convertit $[b_0, b_1, b_2, b_3] \rightarrow "b_0b_1b_2b_3" \rightarrow \text{int}(\dots, 2) \rightarrow$ valeur entre 0 et 15.
3. On applique la S-box : $\text{sub_val} = \text{S_BOX}[\text{nibble_val}]$.
4. On reconvertit sub_val en 4 bits ($f"\{\text{sub_val}:04b}"$).
5. On étend new_block avec ces 4 bits.

Schéma :

```
Bloc : [1,0,1,1, 0,0,1,0, ...]
Nibbles : [1011] [0010] ...

1011 → 11 → S_BOX[11] = 12 → 1100
0010 → 2 → S_BOX[2] = 13 → 1101
...
Bloc transformé : [1,1,0,0, 1,1,0,1, ...]
```

7. ShiftRows : diffusion par réorganisation

```
def shift_rows(block: list[int]) -> list[int]:
    # Décalage des lignes du bloc pour diffusion

    row1 = block[:8]
    row2 = block[8:]

    row2_shifted = row2[1:] + row2[:1] # Décalage circulaire à gauche de 1
    new_block = row1 + row2_shifted
    return new_block
```

- On découpe le bloc (16 bits) en deux “lignes” :
 - row1 = bits 0–7
 - row2 = bits 8–15
- On décale la deuxième ligne d’un cran à gauche (rotation circulaire).

Schéma :

```
Avant :
row1 : r1_0 r1_1 r1_2 r1_3 r1_4 r1_5 r1_6 r1_7
```

```
row2 : r2_0 r2_1 r2_2 r2_3 r2_4 r2_5 r2_6 r2_7
```

Après ShiftRows :

```
row1 : r1_0 r1_1 r1_2 r1_3 r1_4 r1_5 r1_6 r1_7
```

```
row2 : r2_1 r2_2 r2_3 r2_4 r2_5 r2_6 r2_7 r2_0
```

C'est une mini-version de la phase ShiftRows d'AES (où chaque ligne est décalée d'un certain offset).

8. MixColumns : diffusion inter-nibbles

```
def mix_columns(block):  
    # Diffusion très simple et parfaitement inversible :  
    # - découpe en nibbles (4 bits)  
    # - XOR chaque nibble avec une constante (0xA)  
    # - rotation circulaire des nibbles vers la gauche  
    if not block:  
        return []  
  
    if len(block) % 4 != 0:  
        pad_len = 4 - (len(block) % 4)  
        block = block + [0] * pad_len  
  
    nibbles = []  
    for i in range(0, len(block), 4):  
        nib = block[i:i+4]  
        val = int(''.join(str(b) for b in nib), 2) & 0xF  
        nibbles.append(val)  
  
    # XOR with constant  
    xored = [ (n ^ 0xA) & 0xF for n in nibbles ]  
    # rotate left by 1 nibble  
    if len(xored) > 1:  
        xored = xored[1:] + xored[:1]
```

```

new_block = []
for v in xored:
    bits = [int(b) for b in f"{v:04b}"]
    new_block.extend(bits)
return new_block

```

Étapes :

1. Découpage en nibbles (4 bits) → nibbles (liste d'entiers 0–15).
2. XOR de chaque nibble avec 0xA (1010) :

$$\text{xored} = [(n \oplus 0xA) \& 0xF \text{ for } n \text{ in nibbles}]$$
3. Rotation circulaire de la liste de nibbles → diffusion entre les colonnes.
4. Recomposition d'un bloc de bits.

Schéma (exemple avec 4 nibbles) :

Bloc : [n0, n1, n2, n3]

XOR avec 0xA → [n0', n1', n2', n3']

Rotation gauche → [n1', n2', n3', n0']

Recomposition en bits

C'est une version très simplifiée de MixColumns d'AES, mais conçue pour être **facilement inversible**, ce qui est crucial pour le déchiffrement. Contrairement à AES réel, cette fonction ne repose pas sur une multiplication dans $GF(2^8)$. Elle a pour objectif principal d'introduire une diffusion minimale tout en restant simple et parfaitement inversible.

9. Un round complet

```

def round(block, round_key, last_round=False):
    # Un round complet : AddRoundKey → SubBytes → ShiftRows → MixColumns (sauf dernier)
    add_round_keyed = add_round_key(block, round_key)
    subbed = sub_bytes(add_round_keyed)
    shifted = shift_rows(subbed)
    if not last_round:
        mixed = mix_columns(shifted)
        return mixed
    else:

```

```
return shifted
```

Pipeline d'un round :

1. `add_round_key` : injecte la clé de round.
2. `sub_bytes` : confusion locale via S-box.
3. `shift_rows` : diffusion horizontale.
4. `mix_columns` : diffusion verticale (sauf pour le dernier round).

Cette structure est directement inspirée d'AES (sans la partie permutation de lignes et colonnes en 4×4 octets, mais analogique sur 2×8 bits).

10. Chiffrement complet

```
def encrypt(bits, master_key, rounds=ROUNDS):
    # apply PKCS7-like padding (expects bits length multiple of 8)
    if len(bits) % 8 != 0:
        raise ValueError('encrypt expects bits length multiple of 8')
    bits_padded = pad_bits_pkcs7(bits, BLOCK_SIZE)
    blocks = split_blocks(bits_padded, BLOCK_SIZE)
    round_keys = generate_round_keys(master_key, rounds, KEY_SIZE)
    for i in range(len(blocks)):
        block = blocks[i]
        for r in range(rounds):
            last_round = (r == rounds - 1)
            block = round(block, round_keys[r], last_round)
        blocks[i] = block
    # Concaténation des blocs chiffrés
    encrypted_bits = []
    for block in blocks:
        encrypted_bits.extend(block)

    return encrypted_bits
```

Étapes :

1. Vérifie que la longueur en bits est multiple de 8 (car padding PKCS#7 est byte-wise).

2. Applique pad_bits_pkcs7 pour garantir un nombre entier de blocs de 2 octets (16 bits).
3. Découpe en blocs de 16 bits.
4. Génère round_keys.
5. Pour chaque bloc :
 - Applique round rounds fois, avec la bonne sous-clé, en marquant le dernier round.
6. Concatène tous les blocs chiffrés en une seule liste de bits.

Schéma :

bits → padding → split_blocks (16 bits)

→ Block 1 → Round1 → Round2 → Round3 → Round4 → Block1'

→ Block 2 → ...

concaténation → encrypted_bits

11. Déchiffrement bloc + bytes

11.1 decrypt_bits_to_bytes

```
def decrypt_bits_to_bytes(bits_encrypted, master_key, rounds=ROUNDS):  
    # Similar to decrypt but returns raw bytes (no text decoding)  
    blocks = split_blocks(bits_encrypted, BLOCK_SIZE)  
    round_keys = generate_round_keys(master_key, rounds, KEY_SIZE)  
    for i in range(len(blocks)):  
        block = blocks[i]  
        for r in range(rounds - 1, -1, -1):  
            if r != rounds - 1:  
                block = inverse_mix_columns(block)  
                block = inverse_shift_rows(block)  
                block = inverse_sub_bytes(block)  
                block = add_round_key(block, round_keys[r])  
        blocks[i] = block  
  
    decrypted_bits = []  
    for block in blocks:
```

```

        decrypted_bits.extend(block)

# Ensure full bytes
if len(decrypted_bits) % 8 != 0:
    decrypted_bits = decrypted_bits[:len(decrypted_bits) - (len(decrypted_bits) % 8)]
data = bits_to_bytes(decrypted_bits)
# remove PKCS7-like padding (byte-wise)
if len(data) == 0:
    return data
pad_val = data[-1]
bytes_per_block = BLOCK_SIZE // 8
if 1 <= pad_val <= bytes_per_block and all(b == pad_val for b in data[-pad_val:]):
    return data[:-pad_val]
return data

```

Pour chaque bloc, on applique l'inverse des opérations dans l'ordre inverse :

- Encryption round :
AddRoundKey → SubBytes → ShiftRows → MixColumns (sauf dernier round).
- Déchiffrement round :
pour chaque round (r) de rounds-1 à 0 :
 1. Si r != rounds - 1 : inverse_mix_columns.
 2. inverse_shift_rows.
 3. inverse_sub_bytes.
 4. add_round_key (AddRoundKey est auto-inverse).

Enfin :

- On recompose tous les bits.
- On garde un multiple de 8 bits.
- On convertit en bytes.
- On supprime le padding PKCS#7-like, si valide.

12. Fonctions de chiffrement/déchiffrement de fichiers

12.1 encrypt_file

```
def encrypt_file(input_path: str, output_path: str, master_key: int, output_format: str = 'bin'):

    # Read input as binary, encrypt, write either binary or hex

    with open(input_path, 'rb') as f:

        data = f.read()

    bits = bytes_to_bits(data)

    encrypted_bits = encrypt(bits, master_key)

    if output_format == 'hex':

        with open(output_path, 'w', encoding='utf-8') as fo:

            fo.write(bits_to_hex(encrypted_bits))

    else:

        with open(output_path, 'wb') as fo:

            fo.write(bits_to_bytes(encrypted_bits))
```

- Lis le fichier en binaire (rb).
- Convertit les bytes en bits.
- Chiffre les bits.
- Écrit soit :
 - en hex (.write(bits_to_hex(...))) en mode texte,
 - soit en binaire (bits_to_bytes(...)).

12.2 decrypt_file

```
def decrypt_file(input_path: str, output_path: str, master_key: int, input_format: str = 'bin'):

    # Read encrypted input (bin or hex), decrypt and write raw bytes

    if input_format == 'hex':

        with open(input_path, 'r', encoding='utf-8') as f:

            hexstr = f.read().strip()

            bits = hex_to_bits(hexstr)

    else:

        with open(input_path, 'rb') as f:

            data = f.read()

            bits = bytes_to_bits(data)

    decrypted_bytes = decrypt_bits_to_bytes(bits, master_key)

    with open(output_path, 'wb') as fo:
```

```
fo.write(decrypted_bytes)
```

- Lit soit de l'hex, soit du binaire.
- Convertit en bits.
- Déchiffre en bytes.
- Écrit les bytes bruts dans le fichier de sortie.

13. Fonctions inverses (S-box, ShiftRows, MixColumns)

13.1 Inverse S-box

```
INVERSE_S_BOX = {v: k for k, v in S_BOX.items()}
```

- On construit un dict inverse : sortie → entrée.

```
def inverse_sub_bytes(block):  
    # Inverse de sub_bytes (nibble par nibble)  
    new_block = []  
    for i in range(0, len(block), 4):  
        nibble_bits = block[i:i+4]  
        nibble_val = int("".join(map(str, nibble_bits)), 2)  
        orig_val = INVERSE_S_BOX.get(nibble_val, 0)  
        orig_bits = [int(b) for b in f"{orig_val:04b}"]  
        new_block.extend(orig_bits)  
  
    return new_block
```

- Même logique que sub_bytes, mais en passant par INVERSE_S_BOX.

13.2 Inverse ShiftRows

```
def inverse_shift_rows(block: list[int]) -> list[int]:  
    # Inverse du shift_rows: rotation droite de la seconde ligne  
    row1 = block[:8]  
    row2 = block[8:]  
    row2_unshifted = row2[-1:] + row2[:-1]  
    return row1 + row2_unshifted
```

- Rotation droite d'un cran sur la deuxième ligne → inverse exact de la rotation gauche.

13.3 Inverse MixColumns

```
def inverse_mix_columns(block):
```

```

# Inverse of the simple mix_columns above:

# - reconstruct nibbles

# - rotate right by 1 nibble

# - XOR with the same constant (0xA) to recover original

if not block:
    return []

if len(block) % 4 != 0:
    pad_len = 4 - (len(block) % 4)
    block = block + [0] * pad_len

final_nibbles = []
for i in range(0, len(block), 4):
    nib = block[i:i+4]
    val = int(''.join(str(b) for b in nib), 2) & 0xF
    final_nibbles.append(val)

if len(final_nibbles) > 1:
    y = final_nibbles[-1:] + final_nibbles[:-1]
else:
    y = final_nibbles[:]

orig = [ (v ^ 0xA) & 0xF for v in y ]

new_block = []
for val in orig:
    bits = [int(b) for b in f"{val:04b}"]
    new_block.extend(bits)

return new_block

```

1. Reconstruction des nibbles (4 bits).
 2. Rotation droite de 1 nibble (inverse de la rotation gauche).
 3. XOR avec la même constante 0xA : comme $(x \oplus c) \oplus c = x$, on récupère l'original.
-

14. Déchiffrement texte wrapper

```
def decrypt(bits_encrypted, master_key, rounds=ROUNDS):  
    ...  
    decrypted_bytes = decrypt_bits_to_bytes(bits_encrypted, master_key, rounds)  
    try:  
        return decrypted_bytes.decode('utf-8')  
    except Exception:  
        return decrypted_bytes.decode('utf-8', errors='replace')
```

- Cette fonction est un **wrapper haut niveau** pour obtenir directement du texte à partir de bits chiffrés.
 - En pratique, pour les fichiers, tu utilises plutôt `decrypt_file`.
-

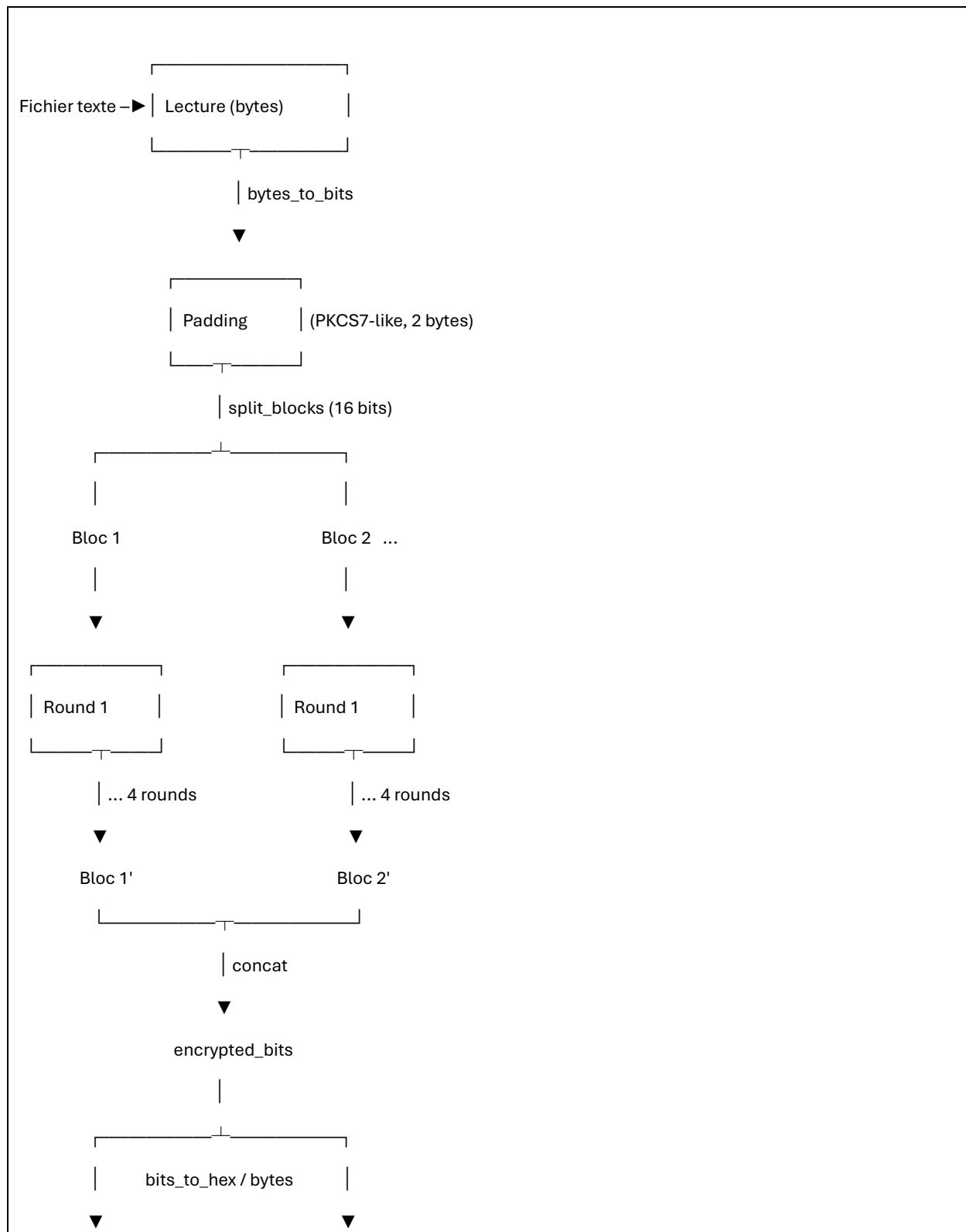
15. Interface interactive main()

```
def main():  
    ...  
    print('AES simple – chiffrement/déchiffrement de fichiers (interactive)')  
    while True:  
        print('\nActions:')  
        print(' 1) Encrypt file')  
        print(' 2) Decrypt file')  
        print(" q) Quit")  
        choice = input('Votre choix: ').strip().lower()  
        ...
```

- Menu simple :
 - 1 → appelle `encrypt_file(...)` sur le fichier texte prédéfini.
 - 2 → appelle `decrypt_file(...)` sur le dernier fichier chiffré hex.
 - q → quitte.
-

16. Vue d'ensemble : pipeline complet

Schéma global :



Fichier hex ou binaire chiffré

Et en sens inverse pour le déchiffrement.

Conclusion

Ce projet de chiffrement **AES simplifié** a été réalisé dans un objectif personnel et pédagogique, afin d'approfondir la compréhension des mécanismes internes d'un chiffrement par blocs moderne, et d'enrichir mon profil GitHub en vue de projets futurs dans les domaines de la cryptographie et de la cybersécurité.

À travers cette implémentation, j'ai pu manipuler concrètement les concepts fondamentaux utilisés par AES réel, tels que la génération de sous-clés (key schedule), les opérations de confusion via une S-box, les mécanismes de diffusion (ShiftRows et MixColumns), ainsi que l'enchaînement de rounds successifs avec AddRoundKey. Le choix de blocs et de clés de 16 bits permet de conserver une complexité maîtrisée tout en rendant le fonctionnement interne du chiffrement lisible et compréhensible.

Le programme prend en charge le chiffrement et le déchiffrement de fichiers binaires complets, en travaillant directement sur des bytes et des bits, avec un padding de type PKCS#7 simplifié, garantissant la réversibilité du processus. Chaque transformation a été conçue pour être explicitement inversible, ce qui permet un déchiffrement correct et cohérent avec la phase de chiffrement.

Il est important de souligner que ce chiffrement n'a aucune vocation à être utilisé en conditions réelles de sécurité. La taille de clé réduite, le nombre limité de rounds et les transformations simplifiées rendent ce système vulnérable à des attaques triviales. L'objectif de ce projet n'est donc pas la sécurité, mais la compréhension des principes cryptographiques sous-jacents.

Ce travail constitue une base solide pour des évolutions futures, telles que l'augmentation de la taille des blocs, l'introduction de structures mathématiques plus proches d'AES réel (champs finis), ou encore la mise en place de modes de chiffrement. Il s'inscrit dans une démarche progressive d'apprentissage et de démonstration technique à travers des projets personnels documentés.